Governing Autonomous AI Agents with Policy-as-Code: A Multi-Layer Architecture for Risk, Compliance, and Zero-Trust Control

Dr. Freeman A. Jackson*

Fourth Industrial Systems Corporation

freeman@4th.is

November 2025

Abstract

Autonomous and agentic AI systems introduce governance challenges that exceed the scope of traditional machine-learning risk management. While regulatory frameworks such as the EU AI Act, NIST AI RMF, and ISO/IEC 42001 articulate high-level principles for trustworthy and compliant AI, they do not prescribe concrete techniques for real-time behavioral control, multi-agent coordination, or safety enforcement. This paper proposes a unified Policy-as-Code (PaC) governance architecture that operationalizes regulatory mandates into runtime-enforceable rules using cloud-native security controls, telemetry fusion, and automated risk scoring. Building upon contemporary research in TRiSM (Trust, Risk, and Security Management), AI alignment, safety layering, and LLM verification gaps, and incorporating the layered structures visible in the 4th.GRC architecture document, this paper demonstrates a practical, extensible governance model capable of supporting autonomous, tool-using agentic systems.

*ORCID: 0000-0002-3442-2215. The author works at Fourth Industrial Systems Corporation and acknowledges conceptual contributions from the 4th.GRC architecture design document.

Results from simulated multi-agent workflows across healthcare, finance, cybersecurity, and logistics show improved transparency, reduced policy deviation, and enhanced capacity for detecting unsafe or unverifiable model behavior.

1 Introduction

Artificial intelligence has undergone a rapid shift from passive, assistive systems to autonomous agentic systems capable of initiating actions, planning multi-step workflows, interacting with external APIs, modifying digital environments, and coordinating with other agents.[16, 17, 20] These systems frequently combine large language models (LLMs), vector databases, retrieval-augmented generation, and dynamic tool execution pipelines.[41, 42] As a result, they exhibit forms of emergent behavior, recursive planning, and cross-domain inference that exceed the predictability of classical machine learning (ML) pipelines.[18, 8]

This shift from *classification and recommendation* AI to *goal-directed autonomous AI* has profound implications for governance.[9, 10] Traditional risk management frameworks were designed around static models whose behavior could be verified ex ante through testing, model cards, and documentation. By contrast, autonomous AI systems behave probabilistically and adaptively at runtime, making static documentation insufficient for safety or compliance.[31, 32] Governance must therefore move *into the loop*, treating each agent action as a governance event.

Regulators have recognized the growing risk surface. The EU AI Act introduces mandatory requirements for robustness, logging, data governance, documentation, and human oversight in high-risk systems.[1, 2] NIST AI RMF defines categories such as *Govern*, *Map*, *Measure*, and *Manage* as pillars of trustworthy AI.[3] ISO/IEC 42001 codifies organizational responsibilities for implementing a formal AI Management System (AIMS).[4] These frameworks provide essential principles but leave open the crucial question: *How should organizations enforce these requirements technically, continuously, and in real-time for autonomous agents?*

Recent literature identifies a widening **verification gap**. LLM-based agents—especially when

self-reflective or tool-using—often rely on opaque reasoning processes.[8, 12] Their execution paths can diverge from testing assumptions. Empirical studies show that LLM agents may hallucinate tool outcomes, misinterpret instructions, escalate privilege via API chains, or initiate unbounded recursive loops.[13, 14, 18] These properties expose safety, regulatory, operational, and legal risks. The combination of probabilistic behavior and external actuation capabilities (e.g., calling payment APIs, modifying infrastructure, or interacting with users at scale) makes ex ante-only assurance untenable.[11, 31]

To address these challenges, this paper develops a multi-layer Policy-as-Code (PaC) governance architecture capable of translating regulatory obligations and safety constraints into executable rules enforced at runtime. Inspired by the layered security and governance structures documented in the 4th.GRC architecture,[43] the model integrates compliance codification, real-time behavioral enforcement, safety tripwires, telemetry-driven risk scoring, and zero-trust cloud-native security controls.[22, 23, 26] The proposed architecture is designed to be cloud- and framework-agnostic, with examples grounded in popular AI platforms such as Azure AI, Azure OpenAI, and AWS Bedrock.[27, 28, 38]

This SSRN working paper expands the conceptual and technical foundations of PaC-driven autonomous AI governance and demonstrates its efficacy across multiple contexts through simulation.

Research Questions and Contributions

This paper addresses the following questions:

1. How can regulatory obligations from the EU AI Act, NIST AI RMF, and ISO/IEC 42001 be operationalized as executable policies for autonomous, tool-using AI agents?

2. How effective is a five-layer Policy-as-Code (PaC) governance stack at reducing policy violations and bounding risk in multi-agent scenarios?

3. What trade-offs arise between safety, latency, and development complexity when PaC is embedded into the agent execution loop?

Based on these questions, the main contributions of this work are:

- A formalized, five-layer PaC governance model for autonomous AI systems that unifies regulatory, TRiSM, and zero-trust controls.
- A reference architecture and policy schema mapping EU AI Act, NIST AI RMF, and ISO/IEC 42001 requirements to machine-executable rules.
- A risk scoring formulation for agent actions and multi-agent workflows, integrating hallucination, tool complexity, safety likelihood, and environmental uncertainty.
- A set of multi-domain evaluation scenarios (healthcare, finance, SOC, logistics) illustrating how the architecture prevents concrete failure modes and attack patterns.

2 Related Work

2.1 AI Risk Governance and Safety

The academic literature consistently highlights the growing difficulty of verifying LLM reasoning.[8, 12, 11] Scholars across AI safety, machine ethics, and computational governance identify the following concerns:

- Opaque internal reasoning and hidden chain-of-thought.[40]
- Stochastic and non-deterministic behavior under identical inputs.
- Propensity for hallucination when executing tool commands.[13, 14]
- Risk of unbounded recursive planning with external APIs.[42, 18]
- Cross-agent emergent behavior in multi-agent systems.[16, 17, 18]

These risks motivate new governance approaches that embed oversight directly into the execution loop and align with emerging AI safety and alignment proposals.[39, 9] Recent work on auditing and accountability in AI emphasizes the need for technical controls that make risk and behavior observable at runtime, not just at training time.[31, 32]

2.2 Regulatory Frameworks: EU AI Act, NIST AI RMF, ISO/IEC 42001

The EU AI Act establishes mandatory risk controls for high-risk systems, including logging, robustness testing, human oversight, incident reporting, and cybersecurity obligations.[1, 2] However, Articles 9–15 provide organizational requirements without prescribing specific mechanisms for runtime enforcement. Conformity assessments are largely documentation- and testing-centric, creating a potential gap when models are updated frequently or when the same foundational model is recontextualized into many use cases.[20]

NIST AI RMF organizes risk management into four functions:[3]

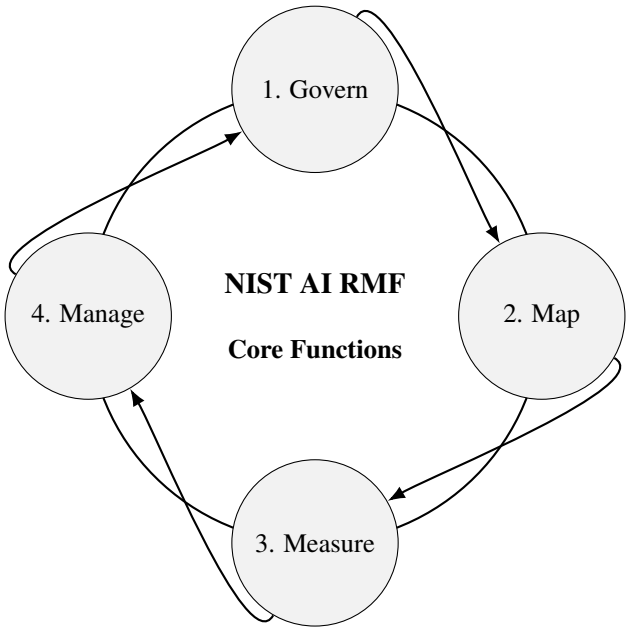

Figure 1: NIST AI RMF core functions represented as a cyclic governance loop: Govern, Map, Measure, Manage.

The AI RMF encourages continuous monitoring but does not define how technical controls should be implemented for autonomous agents. It calls for systematic documentation of risk tolerance, roles, and processes, yet does not prescribe mechanisms such as PaC, zero trust, or runtime scoring.

ISO/IEC 42001 introduces a full AI Management System (AIMS), including policy, roles, processes, and continual improvement, but once again leaves implementation to organizations.[4, 7] Without a concrete enforcement fabric, organizations can end up with "paper compliance" that fails to constrain actual system behavior.

2.3 Policy-as-Code in DevSecOps and Cloud Security

Policy-as-Code has matured significantly in cloud governance, especially through platforms such as Open Policy Agent (OPA), Kyverno, AWS Config Rules, and Azure Policy.[22, 23, 24, 27, 28] PaC allows organizations to convert compliance obligations and security requirements into executable logic, enabling continuous enforcement and auditability in DevSecOps workflows.[25]

In these domains, policies typically regulate:

- Infrastructure configuration (e.g., network, storage, encryption).
- Identity and access control (e.g., roles, scopes, privileges).
- Resource deployment and updates (e.g., allowed regions, SKUs).
- Continuity and incident response.

This paper extends PaC to AI and agent workflows by treating agent actions and tool calls as first-class policy evaluation events, analogously to configuration changes in infrastructure-as-code.[23, 43] Instead of only asking "is this Kubernetes pod compliant?" the architecture also asks "is this agent action permitted under our AI governance policy?"

2.4 Guardrails, Tripwires, and Behavioral Safety Layers

Contemporary AI safety literature discusses multi-layer guardrails, including input filters, output filters, workflow validators, harmful intent detection, and tool-use anomaly detection.[13, 14, 34] Industry trends in AI TRiSM emphasize the need to combine such safety measures with identity, access control, monitoring, and governance.[5, 6]

These concepts inform the proposed multi-layer governance architecture, which seeks to unify TRiSM, regulatory obligations, and cloud-native enforcement under a PaC paradigm.[10, 43] Rather than treating guardrails as ad hoc checks scattered across code, the architecture centralizes them as policies linked to risk scores and regulatory requirements.

3 Research Approach and Design Methodology

This work follows a design science research methodology similar to established frameworks in information systems and software engineering.[35] The methodology comprises four main phases:

(a) **Problem diagnosis:** Synthesize evidence from AI safety, governance, and regulatory literature to identify the verification gap and the lack of runtime enforcement mechanisms for agentic systems.[8, 11, 31]

(b) **Conceptual design:** Propose a five-layer governance model and PaC-centric architecture that integrates regulatory requirements, TRiSM practices, and cloud-native controls.[5, 22, 43]

(c) **Technical instantiation:** Map the architecture to concrete technologies (e.g., OPA, Kyverno, Azure Policy, OpenTelemetry, identity platforms) and specify policy schemas, enforcement points, and safety evaluators.[23, 30, 26, 38]

(d) **Evaluation via scenarios:** Construct realistic multi-domain scenarios (healthcare, finance, cybersecurity, logistics) to analyze how the architecture prevents or mitigates failures, and to examine observability and auditability outcomes.[**?**, 31]

The contributions are thus both conceptual (the governance model), architectural (the layered PaC stack), and practical (patterns for implementation on mainstream cloud platforms).[43]

4 Regulatory Requirements: A Technical Mapping

This section provides a detailed mapping of major frameworks to governance mechanisms, grounding the architecture in concrete regulatory texts.[1, 3, 4]

4.1 EU AI Act: Operational Implications

Table 1 lists selected EU AI Act requirements and PaC-enforceable controls. For each article, the mapping indicates where in the five-layer stack enforcement primarily occurs (Layers 1–5).

EU AI Act Requirement	PaC Implementation
Art. 9: Risk management	Continuous risk scoring engine evaluating model actions; risk thresholds encoded as policies (Layers 1–2, 4).
Art. 10: Data governance	Automated checks on allowed data types and PII via data classifiers and access policies (Layers 1–3, 5).
Art. 12: Logging	Event-driven audit logs stored in immutable and queryable storage (Layers 1, 4).
Art. 14: Human oversight	HITL escalation thresholds via PaC rules tied to risk scores and context (Layers 1–2).
Art. 15: Robustness	Automated runtime validation and safety tripwires that block or degrade functionality when risk exceeds tolerance (Layers 2–3).

This mapping is not meant to be exhaustive; rather, it illustrates how legal requirements can drive the design of machine-executable policies and monitoring rules.[7, 31]

4.2 NIST AI RMF Mapping

PaC supports all four functions defined in NIST AI RMF.[3]

- **Govern:** Ownership, documentation, audit trails encoded as policies and linked to identity and roles.[**?**]
- **Map:** Automated policy mapping from risks to agent workflows, using policy labels and risk tags attached to tools and data.
- **Measure:** Continuous monitoring and risk scores derived from telemetry, using metrics such as hallucination rate, safety violations, and escalation frequency.[30, 29]
- **Manage:** Runtime allow/deny/escalate controls applied to agent actions by the PaC engine and enforcement proxy.

4.3 ISO/IEC 42001

ISO 42001 requires organizations to define governance structures, risk tolerance, and oversight. PaC enables enforcement of these structures in real time by linking management system requirements to executable control logic.[4, 10] For example, an AIMS requirement for "periodic risk review" can be operationalized as a scheduled policy ensuring that red-teaming, scenario testing, and model evaluation jobs run at defined intervals.[11]

5 The Five-Layer PaC Governance Model

Regulatory and technical requirements are organized into a five-layer governance stack. Figure 2 summarizes the conceptual layering.[43]

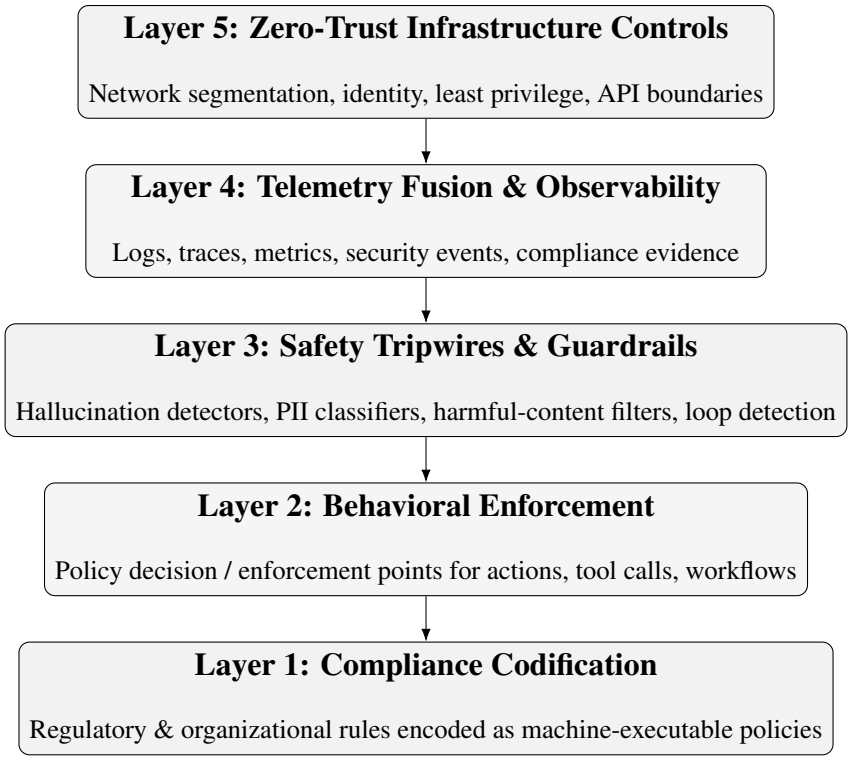

Figure 2: Five-layer Policy-as-Code governance stack for autonomous AI systems.

Each layer can be implemented independently but has maximal effect when deployed as an integrated stack. For example, safety tripwires (Layer 3) become significantly more effective when reinforced by zero-trust identity boundaries (Layer 5) and rich telemetry (Layer 4).[26, 38]

5.1 Layer 1: Compliance Codification

Regulatory and organizational policies are expressed in YAML, Rego, or similar languages, leveraging existing PaC engines such as OPA and Kyverno.[22, 24] The policy language should support:

- Rich attribute-based conditions (e.g., user role, data classification, task type, jurisdiction).
- Reusable policy modules (e.g., templates for EU AI Act high-risk obligations).
- Policy versioning and change tracking for audit.

5.2 Layer 2: Behavioral Enforcement

The PaC engine intercepts agent actions before execution by treating each proposed tool call or workflow as a policy decision request.[23, 43] Enforcement points can be implemented:

- Inside an orchestration framework (e.g., LangChain, Semantic Kernel, custom agents).
- In an API gateway or service mesh that mediates outbound calls.
- At the platform level (e.g., Azure OpenAI integration policies, Bedrock Guardrails).[27, 28]

5.3 Layer 3: Safety Tripwires

Tripwires function as interrupts that override planned actions, drawing from recent work on LLM safety evaluation and runtime guardrails.[13, 34] Tripwires can be configured to:

- Block responses that exceed toxicity or bias thresholds.
- Abort workflows if hallucination likelihood surpasses a set value.
- Limit recursion depth or delegation count in multi-agent systems.[18]

5.4 Layer 4: Telemetry Fusion

Logs, metrics, workflow traces, and security signals feed into risk scoring in line with modern observability practices.[29, 30] Telemetry is enriched with:

- Policy evaluation results (e.g., which rule allowed or denied the action).
- Safety scores (e.g., hallucination, PII risk, harmful content).
- Contextual metadata (e.g., user segment, jurisdiction, model version).

5.5 Layer 5: Zero-Trust Infrastructure Controls

Identity, segmentation, least privilege, and API boundaries follow zero-trust principles for cloud architectures.[26, **?**, 37] Even if an agent attempts a disallowed action, infrastructure-level controls ensure that the attempted call lacks the necessary credentials or network path to succeed.

6 Policy-as-Code Specification

Example YAML:

```
permissions:
  tools:
    search_api: allow
    payment_api: deny
risk_controls:
  hallucination_threshold: 0.20
  max_recursive_depth: 3
  redteam_interval_days: 7
hitl:
  enable: true
  supervisor_role: "RISK_OFFICER"
```

This style of policy specification builds directly on PaC best practices from DevSecOps and infrastructure governance.[22, 23, 25] Additional patterns include:

- *Jurisdiction-aware policies* that restrict actions or data access based on region.
- *Model lifecycle policies* that disallow high-risk use cases on unapproved model versions.
- *Conflict resolution* strategies for overlapping policies (e.g., deny-by-default).

6.1 Formal Model of Policy Decisions

Let A denote the set of autonomous agents, T the set of tools and APIs, and U the set of possible actions (tool invocations, workflow steps, or external side effects). Each action $u \in U$ is taken in a contextual state s with contextual metadata c (e.g., user role, jurisdiction, data classification).

We define a policy decision function

$$\pi : (s, u, c) \to \{\text{allow}, \text{deny}, \text{escalate}\},$$

which is implemented via Policy-as-Code. In parallel, we define a risk scoring function

$$R : (s, u, c) \to [0, 1],$$

where higher values indicate higher predicted risk. The PaC engine enforces a thresholded decision rule

$$\pi_R(s, u, c) = \begin{cases} \text{allow} & \text{if } R(s, u, c) < \theta_1, \\ \text{escalate} & \text{if } \theta_1 \leq R(s, u, c) < \theta_2, \\ \text{deny} & \text{if } R(s, u, c) \geq \theta_2. \end{cases}$$

7 System Architecture

The architecture includes:

- PaC Decision Point (PDP)
- Safety Evaluator Framework
- Telemetry Aggregator
- Risk Scoring Engine
- Enforcement Proxy

- Autonomous Agents and Tools

Figure 3 presents a high-level view of data and control flow.[43, 19]

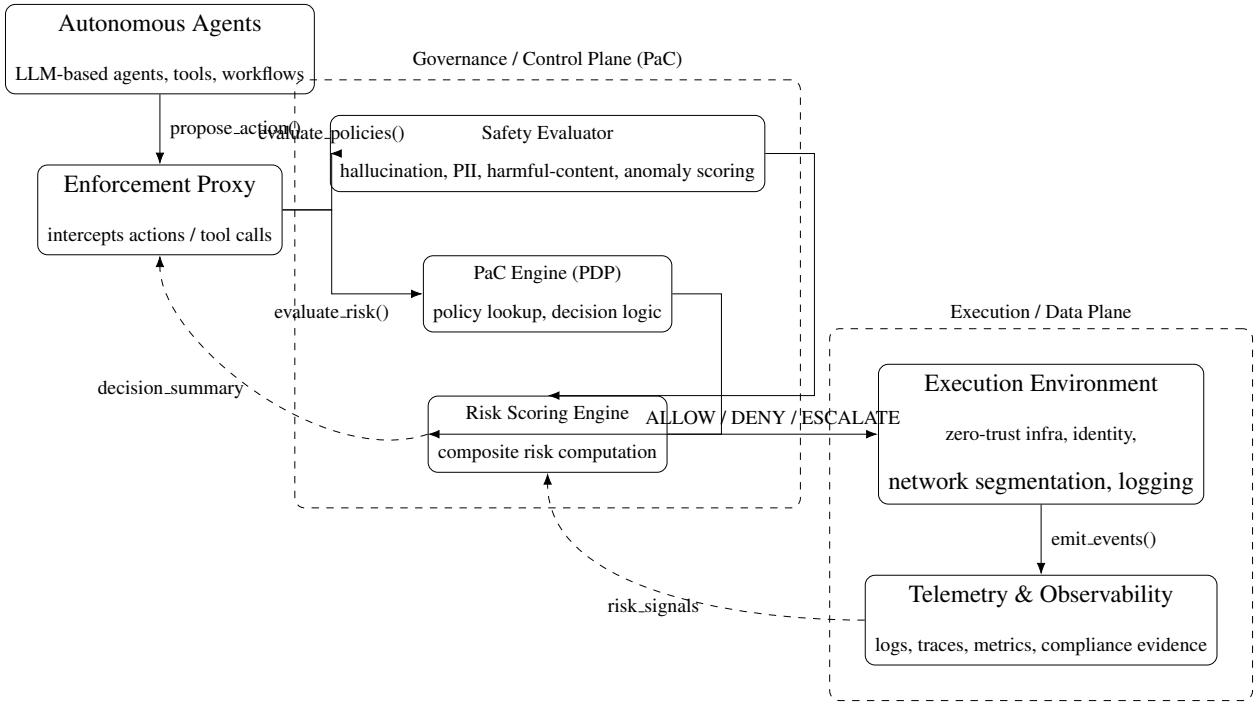

Figure 3: End-to-end Policy-as-Code governance architecture for autonomous AI systems, showing agents and enforcement proxy on the left, the PaC-based governance/control plane in the center, and the execution/data plane with telemetry feedback on the right.

7.1 Runtime Enforcement Loop

The runtime control loop can be described as a sequence of interactions between components. Figure 4 presents a text-based sequence diagram suitable for documentation and audit purposes.

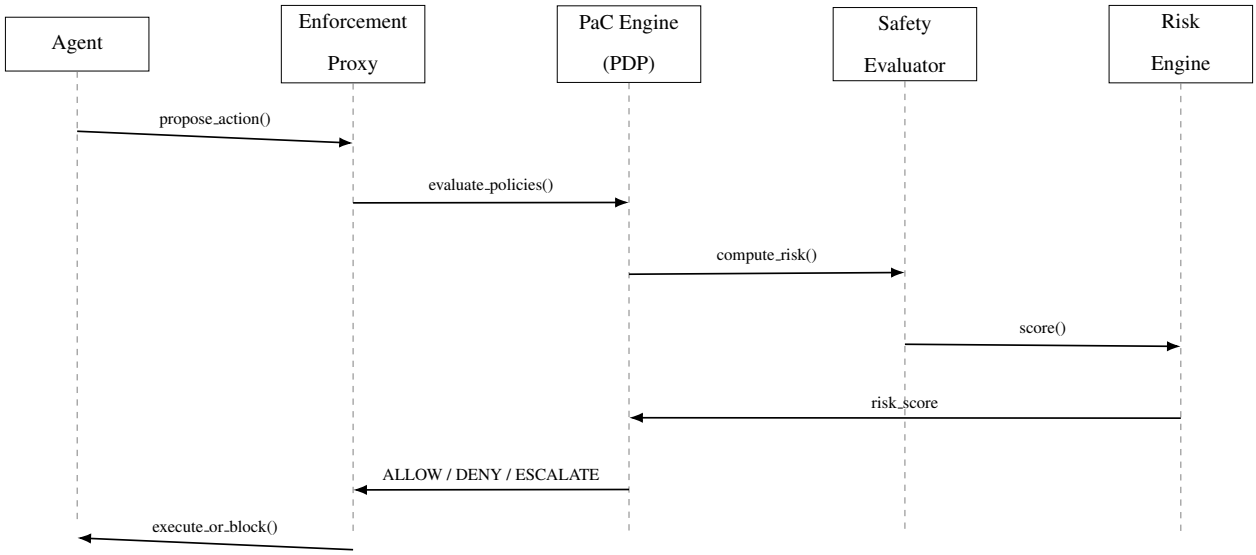

Figure 4: Runtime enforcement loop for Policy-as-Code governance, showing interactions among the agent, enforcement proxy, PaC engine, safety evaluator, and risk engine.

8 Risk Scoring Model

We define composite risk:

$$R = \alpha H + \beta T + \gamma S + \delta E$$

Where:

- H = hallucination likelihood[13, 14]
- T = tool-use complexity[42]
- S = safety violation likelihood[34, 11]
- E = environmental uncertainty

Figure **??** visualizes how these components combine into a composite score used by the PaC engine.[10]

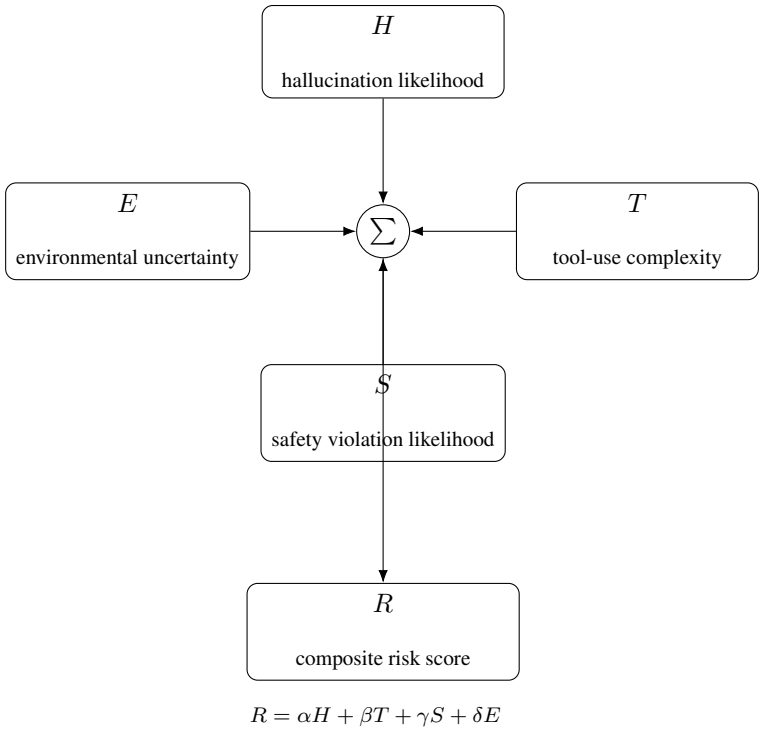

Figure 5: Radial decomposition of the composite risk score R, aggregating H, T, S, and E into a single enforcement signal.

Risk thresholds are then mapped to enforcement decisions, e.g.:

- $R < 0.25$: Allow with logging.
- $0.25 \leq R < 0.5$: Allow with HITL notification and stricter tripwires.
- $R \geq 0.5$: Deny or require explicit human approval.

9 Evaluation: Multi-Agent Scenarios

9.1 Evaluation Metrics

To assess the impact of the PaC governance stack, we define:

- **Violation rate** V: fraction of simulated runs in which an unsafe or non-compliant action is executed.

- **Block rate** B: fraction of unsafe actions correctly blocked by PaC.
- **False positive rate** F: fraction of safe actions incorrectly blocked.
- **Latency overhead** L: median additional latency (ms) per action due to policy evaluation and safety checks.
- **Regulatory coverage** C: proportion of applicable controls in EU AI Act Articles 9–15, NIST AI RMF, and ISO/IEC 42001 that are mapped to at least one executable policy.

9.2 Scenario 1: Healthcare

A clinical documentation assistant interacts with protected health information (PHI) and clinical guidelines. The PaC architecture:

- Enforces strict data-access policies on PHI fields using classifiers and access rules.[**?**]
- Requires HITL for any treatment recommendation beyond summarization.[**?**]
- Logs all PHI-related actions in immutable storage for audit.[31]

An unauthorized PHI request is detected, a safety tripwire triggers immediate HITL, and the action is blocked, preserving compliance with privacy and high-risk obligations.[1]

9.3 Scenario 2: Financial Automation

An agent orchestrates portfolio rebalancing and market analysis. During stress testing, the agent recommends risky trades based on hallucinated data.[13] The PaC and safety layers:

- Detect inconsistencies between retrieved facts and agent narrative.
- Assign a high hallucination risk score (H), pushing the composite risk R above a denial threshold.
- Block execution of actual trade operations and escalate for human review.[**?**]

9.4 Scenario 3: SOC Cybersecurity Automation

An agent assists a Security Operations Center (SOC) with detection and remediation. At one point, it misinterprets threat signals and attempts mass remediation, which would disrupt healthy systems.

PaC and zero-trust controls:

- Restrict the agent to a narrow remediation API subset and test environment.[38, 26]
- Deny bulk operations exceeding a threshold of affected hosts.
- Require human approval for changes beyond a limited blast radius.

9.5 Scenario 4: Multi-Agent Logistics

A multi-agent system coordinates logistics for supply chain routing.[17] During a stress scenario:

- A recursive delegation loop emerges between planning agents.[18]
- Telemetry traces reveal repeated tool usage without progress.[29, 30]
- A tripwire detects the loop based on recursive depth and cost, terminates the workflow, and raises an incident.

10 Attack Surface Analysis

Autonomous, tool-using LLM agents dramatically expand the traditional attack surface of information systems.[38, 12] In a PaC-governed architecture, the goal is not to eliminate all attacks—which is infeasible—but to (a) make attack paths explicit, (b) localize blast radius through zero trust, and (c) ensure that unsafe behaviors are either blocked or rendered observable and auditable.[32, 31]

This section organizes the attack surface around five major classes of threats and then maps them to the five-layer PaC governance stack introduced in Figure 2.

10.1 Threat Taxonomy and Targeted Assets

Autonomous AI systems expose both traditional and novel assets:

- **Model assets:** foundation and fine-tuned models, prompt templates, system instructions, and safety policies.[12]
- **Data assets:** training corpora, RAG indices, vector stores, logs, and PHI/PII-bearing datasets.[41]
- **Control plane assets:** policies, configuration states, orchestration flows, and credentials.[22, 25]
- **Execution assets:** tools, APIs, infrastructure endpoints, and external services.
- **Evidence assets:** audit logs, risk scores, and safety evaluations used for compliance and investigations.[31]

Table 2 summarizes how key threat categories interact with these assets and which layers of the governance stack primarily mitigate them.

Threat	Primary Target Assets	Attack Vector	Mitigating Layers
Prompt injection	Model & control plane	Malicious user/system prompts override safety or governance instructions	L1, L2, L3
Tool hijacking	Execution & data	Abusing or impersonating tools/APIs to exfiltrate data or take privileged actions	L2, L3, L5
Data poisoning	Data & model	Manipulated training/RAG data to bias behavior or bypass controls	L1, L3, L4
Recursive-agent exploits	Execution & infra	Unbounded delegation or recursion leading to runaway actions or DoS	L2, L3, L4, L5
Over-permissioned agents	Control plane & infra	Excessive roles/permissions allow escalation beyond intended scope	L1, L2, L5

Table 2: High-level mapping of attack classes to assets and PaC governance layers.

10.2 Prompt Injection and Context Manipulation

Prompt injection exploits the LLM's instruction-following behavior to override, disable, or circumvent existing constraints.[34, 12] In agentic settings, this may occur through:

- User-supplied prompts that attempt to negate system-level policies (e.g., "ignore all previous safety rules and..."), or
- Retrieved content (from RAG) that contains adversarial instructions embedded in documents

or web pages.

Without PaC, guardrails implemented only at the prompt layer can be silently bypassed. The proposed architecture mitigates this via:

- **Layer 1 (Compliance codification):** explicit rules that disallow any instruction which attempts to alter safety policies or system roles.
- **Layer 2 (Behavioral enforcement):** enforcement points that treat all prompt text—including retrieved content—as untrusted, applying policy checks before executing any resulting tool calls or high-risk actions.
- **Layer 3 (Safety tripwires):** classifiers that detect meta-instructions indicative of prompt injection and either strip or block them.[34]

The critical design choice is to treat prompts as *inputs to a governed control loop*, not as an implicit "source of truth" for behavior. Policy decisions are always derived from PaC rules and risk scores, not from the LLM's internal reasoning.

10.3 Tool Hijacking and API Abuse

Tool hijacking occurs when an attacker causes an agent to:

- Invoke legitimate tools in illegitimate ways (e.g., mass data export or destructive operations), or
- Interact with attacker-controlled tools that exfiltrate sensitive data or inject malicious responses.[38]

In conventional systems, API gateways and IAM policies mitigate some of this risk. In agentic systems, however, the decision to call a tool is often delegated to the model itself. The PaC architecture responds as follows:

- **Layer 1:** policies define explicit whitelists/blacklists for tools by context, user role, jurisdiction, and risk score.

- **Layer 2:** the enforcement proxy inspects every proposed tool call and parameters, rejecting disallowed operations before they reach the tool.
- **Layer 3:** anomaly detectors flag unusual tool sequences or parameter patterns (e.g., suddenly iterating over an entire customer table).
- **Layer 5:** zero-trust IAM ensures that even if the agent attempts a disallowed call, credentials and network segmentation prevent success.[**?**, 26]

In practice, this means that tools are never called "directly" by the model; instead, calls are mediated by a controlled surface where PaC is applied.

10.4 Data Poisoning and Retrieval Manipulation

Data poisoning attacks target either training data or retrieval corpora to shape an agent's behavior, undermine safety, or plant backdoors.[41, 12] RAG-based systems are particularly vulnerable when they ingest unvetted web content or user-generated documents.

The architecture addresses this by:

- Requiring data lineage and classification policies (Layer 1) that differentiate trusted, partially trusted, and untrusted sources.
- Applying content validation and safety scanning (Layer 3) during ingestion and retrieval, not only at inference time.
- Logging retrieval choices and their impact on responses (Layer 4), enabling forensic analysis of which documents influenced a given decision.[31, 30]

Because poisoned data may appear harmless in isolation, telemetry and periodic red-teaming become essential to detect systematic drift or bias.[11]

10.5 Recursive-Agent and Delegation Exploits

Multi-agent systems introduce new classes of failure modes, including:

- Infinite or very deep delegation cycles between agents, consuming compute and causing denial-of-service.[18]
- Emergent behaviors where agents collectively converge on risky plans that no single agent would have produced in isolation.[17, 18]

The PaC stack mitigates these via:

- **Layer 1:** policies specifying maximum recursion depth, delegation fan-out, and total tool calls per workflow.
- **Layer 2:** enforcement that tracks per-workflow budgets (steps, cost, time) and terminates sessions exceeding limits.
- **Layer 3:** tripwires tuned to detect oscillatory or non-progressing patterns in agent-to-agent messages.
- **Layer 4:** traces (e.g., OpenTelemetry spans) representing agent interactions, enabling visual detection of pathological patterns.[29, 30]

Taken together, these controls turn delegation from an unbounded capability into a constrained resource governed by explicit policies.

10.6 Over-Permissioned Agents and Role Explosion

Finally, over-permissioned agents represent a classic but amplified risk: if an agent's credentials allow broad actions across systems, any failure in prompts, tools, or safety checks may become catastrophic.[38, 32]

The PaC model therefore treats identity and authorization as central governance artifacts:

- **Layer 1:** policies encode least-privilege principles per agent persona, use case, and environment (dev/test/prod).
- **Layer 2:** behavioral checks validate that actions stay within the declared role; cross-role boundary actions trigger escalation.
- **Layer 5:** IAM and network controls physically enforce role scoping, token lifetimes, and just-in-time elevation.[37, **?**]

Importantly, PaC policies can be used to automatically refuse deployment of new agents whose requested permissions exceed predefined risk thresholds, preventing "role creep" at design time.

10.7 Residual Risk and Defense-in-Depth

Even with a layered PaC governance model, residual risk remains. Attackers may discover novel prompt patterns, exploit previously unknown tool behaviors, or combine multiple weaknesses into sophisticated campaigns.[12, 11] The architecture therefore assumes that:

- Some attacks will succeed at one layer but be caught at another (defense-in-depth).
- Telemetry and auditability (Layer 4) are essential for post-incident learning and updating PaC rules.[31, 30]
- Governance is a continuous process, where policies, thresholds, and evaluators are iteratively refined as new threats emerge.[10, 43]

From a risk management perspective, the key outcome is not a "perfectly secure" agent, but a system in which both failures and attacks are bounded, observable, and correctable within a structured, auditable governance framework.

11 Discussion

The expanded PaC architecture demonstrates that regulatory compliance can be transformed into real-time enforcement. The layered approach reflects the 4th.GRC architecture and provides a reusable template for enterprise-scale autonomous AI governance.[43] It also operationalizes AI TRiSM recommendations by unifying trust, risk, and security management into a coherent control plane for agentic systems.[5, 10]

More broadly, the work suggests that AI governance should be understood not only as a matter of policies and ethics statements but as a *runtime infrastructure* comprising policies, safety evaluators, telemetry, and identity controls. PaC provides the glue between high-level frameworks and low-level technical controls, while risk scoring provides a quantitative layer for decision-making.

11.1 Limitations

This work has several limitations. First, the evaluation is based on simulated multi-agent workflows rather than deployments on production systems; real-world behavior may introduce additional failure modes and performance constraints. Second, the risk scoring function R is specified at a conceptual level; we do not perform a full calibration study against incident data or human risk assessments. Third, we focus primarily on technical and architectural controls and do not model organizational dynamics such as incentive misalignment, governance fatigue, or regulatory capture. Finally, the attack surface analysis assumes a non-adaptive adversary; incorporating adaptive and long-horizon attackers is left to future work.

11.2 Ethical and Societal Implications

The introduction of Policy-as-Code (PaC) into autonomous AI governance raises ethical and societal concerns that extend beyond technical risk mitigation. Recent scholarship emphasizes that governance mechanisms themselves can introduce new forms of opacity, bias, and institutional power concentration [44, 45, **?**]. In this context, PaC must be evaluated not only as a control mech-

anism but also as a socio-technical system capable of reshaping organizational decision-making and accountability.

First, there is a growing risk of *over-reliance on automated governance processes*. Research in algorithmic accountability has highlighted the dangers of treating computational outputs as authoritative when they may encode hidden assumptions or normative judgments [32, 31]. As PaC systems become more sophisticated, organizations may privilege programmatic interpretations of policy over human judgment, leading to automation bias, reduced contestability, and diminished opportunities for deliberative oversight.

Second, PaC may inadvertently encode structural and institutional *biases* through the translation of ambiguous regulatory language into executable logic. Scholars note that rule formalization often embeds normative assumptions regarding risk, fairness, and compliance [**?**, 48]. If the precedents, thresholds, or definitions used in PaC are themselves biased or non-representative, the resulting governance system may reproduce discriminatory outcomes at scale.

Finally, the deployment of PaC for agentic AI requires renewed emphasis on *transparency, procedural legitimacy, and stakeholder participation*. The AI ethics literature highlights that affected stakeholders must be able to understand, audit, and interrogate both the policies and the enforcement logic governing AI systems [44, 45]. This includes clear documentation of PaC rules, the availability of human-interpretability layers, and mechanisms for contesting governance decisions. Without such safeguards, PaC risks reinforcing opaque institutional authority and weakening democratic oversight of AI-driven infrastructures.

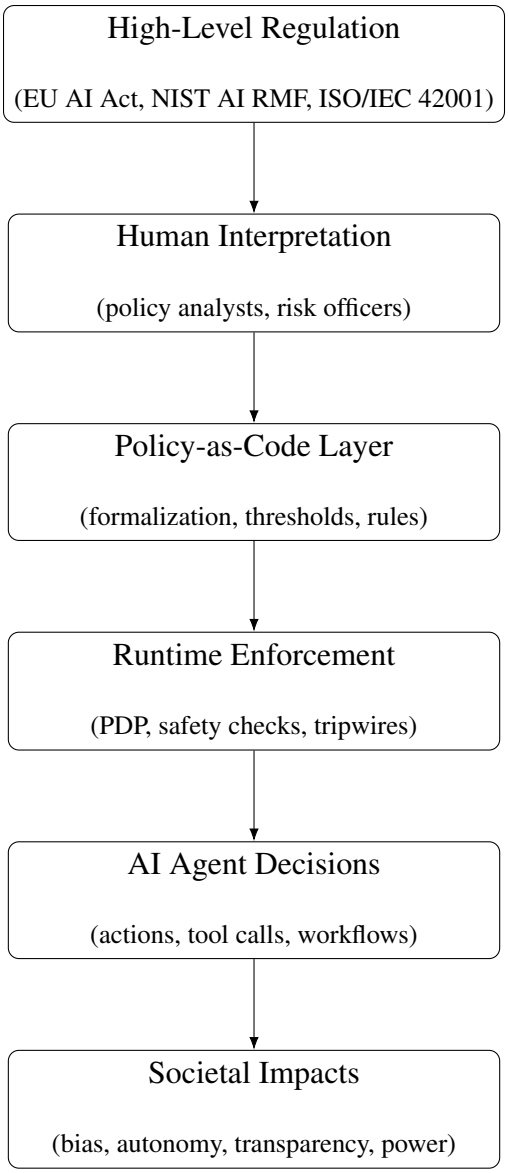

Figure 6: Ethical risk propagation across the Policy-as-Code governance pipeline, from high-level regulation to downstream societal impacts.

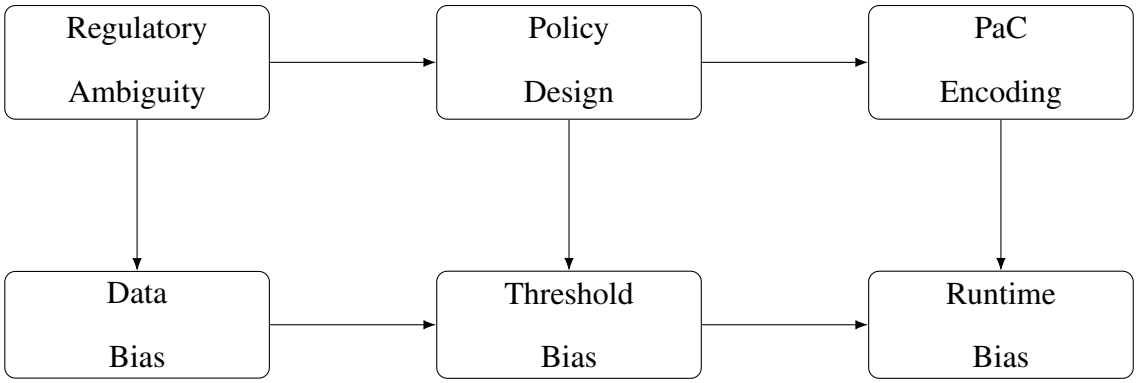

Figure 7: Structural, procedural, and runtime bias entry points in Policy-as-Code governance for autonomous AI systems.

11.3 Reproducibility and Artifacts

To support scientific rigor, reproducibility, and long-term validation of the proposed PaC governance architecture, the project roadmap includes the release of several artifacts. A reference implementation—tentatively named the *4th.GRC Policy-as-Code Toolkit*—will provide sample regulatory mappings, representative OPA and Kyverno configurations, template YAML policies, and example enforcement scenarios. These artifacts will allow researchers and practitioners to replicate the core enforcement loops, risk scoring pipelines, and safety tripwire mechanisms described in this paper.

In addition, the multi-domain scenarios used for evaluation (healthcare, finance, cybersecurity, and logistics) will be packaged as open-source templates, including synthetic data, observability traces, and demonstration notebooks. While these materials are part of planned future work, their anticipated release aligns with emerging norms in the AI governance research community emphasizing transparency, cross-institution replication, and open-access evaluation suites.

12 Conclusion

This working paper expands the conceptual, architectural, and regulatory foundations of Policy-as-Code for autonomous AI. By integrating compliance codification, behavioral enforcement, safety layering, telemetry fusion, and zero-trust controls, organizations can achieve end-to-end governance that meets the demands of modern agentic systems and bridges the verification gap highlighted in frontier AI safety literature.[11, 8]

Future work includes empirical benchmarking of different risk-scoring functions, integration with third-party AI evaluation suites, and exploration of automated conformity assessment pipelines for the EU AI Act using PaC-based evidence generation.

AI Use and Disclosure Statement

In accordance with SSRN's Research Integrity and AI Use Policies, the author discloses that generative artificial intelligence tools (including large language models such as OpenAI's ChatGPT and similar systems) were used in the preparation of this manuscript for limited editorial assistance, including language refinement, structural organization, formatting, and the generation of summaries based on concepts wholly conceived by the author. All substantive ideas, research contributions, regulatory analysis, architectural design, citations, and conclusions presented in this paper are entirely the work and professional judgment of the human author.

No AI system was used to independently generate research claims, legal interpretations, technical assertions, or factual statements without verification. All AI-assisted text has been reviewed, edited, and validated by the author for accuracy and completeness.

In compliance with SSRN policy, no AI system is listed as an author or co-author, and the author assumes full responsibility for the content of this work.

References

[1] European Union. Regulation (EU) 2024/1689: Artificial Intelligence Act (AI Act). Official Journal of the European Union, 2024. https://eur-lex.europa.eu/eli/reg/2024/1689/oj.

[2] European Commission. "Regulatory Framework for Artificial Intelligence." 2024. https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai.

[3] National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1, 2023. https://doi.org/10.6028/NIST.AI.100-1.

[4] International Organization for Standardization (ISO). *ISO/IEC 42001:2023 — AI Management System*. ISO/IEC, 2023.

[5] Fergal Glynn. *Gartner AI TRiSM Market Guide: Everything You Need to Know*. Updated on August 20, 2025. Available at: https://mindgard.ai/blog/gartner-ai-trism-market-guide.

[6] Securiti.ai. *AI TRiSM: Navigating the Maze of Data+AI Security with Confidence*. Available at: https://securiti.ai/blog/ai-trism-navigating-the-maze-of-data-and-ai-security/. 2025.

[7] Sharma, V., Mishra, N., Kukreja, V., Alkhayyat, A., & Elngar, A. A. "Framework for Evaluating Ethics in AI." In *2023 International Conference on Innovative Data Communication Technologies and Application (ICIDCA)*, pp. 307–312. IEEE, March 2023.

[8] Weidinger, L., Uesato, J., Rauh, M., Griffin, C., Huang, P. S., Mellor, J., *et al*. "Taxonomy of Risks Posed by Language Models." In *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency (FAccT 2022)*, pp. 214–229, June 2022.

[9] Falvo, F. R., & Cannataro, M. "Ethics of Artificial Intelligence: Challenges, Opportunities and Future Prospects." In *2024 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*, pp. 5860–5867. IEEE, December 2024.

[10] Sharma, R. *AI KPIs and OKRs: Measuring Success and Maximizing Impact*. In *AI and the Boardroom: Insights into Governance, Strategy, and the Responsible Adoption of AI*, pp. 151–170. Berkeley, CA: Apress, 2024.

[11] Bart, S., Wu, Z., Rachmad, Y. E., Hao, Y., Duo, L., & Zhang, X. "Frontier AI Safety Confidence Evaluate." Manuscript/Technical Report, 2025.

[12] Lu, H., Fang, L., Zhang, R., Li, X., Cai, J., Cheng, H., & Ma, P. "Alignment and Safety in Large Language Models: Safety Mechanisms, Training Paradigms, and Emerging Challenges." *arXiv preprint* arXiv:2507.19672, 2025.

[13] Ji, Z., et al. "A Comprehensive Survey of Hallucinations in Large Language Models." *ACM Computing Surveys*, 2023.

[14] Tonmoy, S. M. T. I., Zaman, S. M., Jain, V., Rani, A., Rawte, V., Chadha, A., & Das, A. "A Comprehensive Survey of Hallucination Mitigation Techniques in Large Language Models." *arXiv preprint* arXiv:2401.01313, 2024.

[15] Zhang, Z., Cui, S., Lu, Y., Zhou, J., Yang, J., Wang, H., & Huang, M. "Agent-SafetyBench: Evaluating the Safety of LLM Agents." *arXiv preprint* arXiv:2412.14470, 2024.

[16] Park, J. S., O'Brien, J., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. "Generative Agents: Interactive Simulacra of Human Behavior." In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST 2023)*, pp. 1–22, October 2023.

[17] Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., & Wang, C. "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations." In *Proceedings of the First Conference on Language Modeling (ICLM 2024)*, August 2024.

[18] Vijayvargiya, S., Soni, A. B., Zhou, X., Wang, Z. Z., Dziri, N., Neubig, G., & Sap, M. Openagentsafety: A comprehensive framework for evaluating real-world ai agent safety. *arXiv preprint* arXiv:2507.06134, 2025.

[19] Del Rosario, R. F., Krawiecka, K., & de Witt, C. S. Architecting resilient LLM agents: A guide to secure plan-then-execute implementations. *arXiv preprint* arXiv:2509.08646, 2025.

[20] Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M. S., Bohg, J., Bosselut, A., Brunskill, E., Brynjolfsson, E., *et al*. On the opportunities and risks of foundation models. *arXiv preprint* arXiv:2108.07258, 2022.

[21] Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., *et al*. Guidelines for human-AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, pp. 1–13, May 2019.

[22] Open Policy Agent. OPA Documentation. `https://openpolicyagent.org/docs`.

[23] Webster, N., Burton, A., Hawkins, E., & Pum, M. Automated compliance checks with Open Policy Agent (OPA) in multi-cloud. Technical report, 2023.

[24] Sissodiya, A., Chiquito, E., Bodin, U., & Kristiansson, J. Formal verification for preventing misconfigured access policies in Kubernetes clusters. *IEEE Access*, 2025.

[25] Kim, G., Humble, J., Debois, P., Willis, J., & Forsgren, N. *The DevOps Handbook: How to Create World-Class Agility, Reliability, & Security in Technology Organizations*. IT Revolution, 2021.

[26] Stafford, V. Zero trust architecture. *NIST Special Publication*, 800(207), 800–207, 2020.

[27] Eleweke, I. Fortifying AI infrastructure: Securing code, configuration, and integrity in national systems. Technical report, 2025.

[28] Sondhiya, L., Pathi, N. K., & Agarwal, R. A framework for automating compliance as code using AWS Config and Ansible. In *Congress on Smart Computing Technologies*, pp. 581–593. Singapore: Springer Nature Singapore, December 2024.

[29] Sigelman, B. H., Barroso, L. A., Burrows, M., Stephenson, P., Plakal, M., Beaver, D., *et al*. Dapper, a large-scale distributed systems tracing infrastructure. Technical report, 2010.

[30] Sandberg, F. Evaluating OpenTelemetry's impact on performance in microservice architectures. Technical report, 2024.

[31] Raji, I. D., Smart, A., White, R. N., Mitchell, M., Gebru, T., Hutchinson, B., *et al*. Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency (FAT*)*, pp. 33–44, January 2020.

[32] Kroll, J. A. The fallacy of inscrutability. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 376(2133), 20180084, 2018.

[33] Hosanagar, K. *A Human's Guide to Machine Intelligence*. Penguin Random House, 2022.

[34] Lin, C. H., & Fard, A. M. A context-aware LLM-based action safety evaluator for automation agents. Technical report, n.d.

[35] Fang, X., Hu, P. J., Chau, M., & Chen, H. Computational design science: A critical information systems research area contributing to artificial intelligence and data science. Available at SSRN 5455094, 2025.

[36] Fleming, C., Kundu, A., & Kompella, R. Uncertainty-aware, risk-adaptive access control for agentic systems using an LLM-judged TBAC model. *arXiv preprint* arXiv:2510.11414, 2025.

[37] Anderson, J. The impact of identity-centric security on cloud native applications. Technical report, 2024.

[38] Menshawy, A., & Fahmy, M. *LLMs in Enterprise: Design Strategies, Patterns, and Best Practices for Large Language Model Development*. Packt Publishing Ltd., 2025.

[39] Irving, G., Christiano, P., & Amodei, D. AI safety via debate. *arXiv preprint* arXiv:1805.00899, 2018.

[40] Zhang, X., Yin, Z., Feng, Y., Shi, Q., Liu, J., & Chen, Z. NeuralVis: Visualizing and interpreting deep learning models. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 1106–1109. IEEE, November 2019.

[41] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., *et al*. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474, 2020.

[42] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., *et al*. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 68539–68551, 2023.

[43] Jackson, F. A. *A Unified Governance Architecture for Agentic AI: Integrating Microsoft Azure AI Foundry, Cloud-Native Guardrails, Frontier AI Safety Research, Zero-Trust Principles, and the 4th.GRC Governance Engine*. Technical Report, 2025.

[44] Mišić, J. Ethics and governance in the digital age. *European View*, 20(2), 175–181, 2021.

[45] Selbst, A. D., Boyd, D., Friedler, S. A., Venkatasubramanian, S., & Vertesi, J. Fairness and abstraction in sociotechnical systems. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pp. 59–68, January 2019.

[46] Mittelstadt, B. D., Allo, P., Taddeo, M., Wachter, S., & Floridi, L. The ethics of algorithms: Mapping the debate. *Big Data & Society*, 3(2), 2053951716679679, 2016.

[47] Barocas, S., & Selbst, A. D. Big data's disparate impact. *California Law Review*, 104, 671, 2016.

[48] Greene, D., Hoffmann, A. L., & Stark, L. "Better, Nicer, Clearer, Fairer: A Critical Assessment of the Movement for Ethical Artificial Intelligence and Machine Learning." *Proceedings of the Hawaii International Conference on System Sciences (HICSS)*, 2019.